

# **Building Risk Models to Predict Railroad Accident Severity**

Tanner Hilt

Master of Data Science, Bryant University

**Abstract:**

The question evaluated is whether it is possible to develop a risk model that helps professionals in the railroad industry determine the severity of an accident involving equipment. Data were used from 1975 to 2026, with over 150,000 records. Two target variables, risk score and risk category, were engineered using a combination of methods, including PCA-weight extraction and Yeo-Johnson Power Transformation. The models implemented were XGBoost Regressor and Classifier against dummy baseline models for the paired t-test and bootstrap resampling. Concluding that the hypothesis is confirmed, as both models showed statistical significance, with scores including R-squared of 0.287 vs - 0.002 and F1-macro of 0.675 vs 0.503. However, the models did not capture a large majority of the data. Therefore, these models are intended to support decision-making rather than final decisions.

**Introduction:**

In the transportation industry, risk is an unavoidable problem that puzzles many professionals to this day. Though no tool can predict the future, building risk models to help provide insight is most important. One risk that is crucial to determining its severity is railroad equipment accidents. Currently, risk assessment plans are completed through manual review, requiring extensive time for evaluation. With data provided from the FRA Safe Team on Data.Transportation.gov regarding the “FRA 6180.54-Rail Equipment Accident/ Incident” form, the objective is to build a predictive and classification model to help professionals quickly gather information about an accident in real time to assess risk and support mitigation strategy reasoning (Office, 2021).

Research Question: Is it possible to develop a risk model that helps professionals in the railroad industry determine the severity of an accident/ incident involving equipment?

Hypothesis: Pre-incident features from the FRA Form 6180.54 data source provide enough information to establish risk scores and classifications for railroad accidents more effectively than a dummy model.

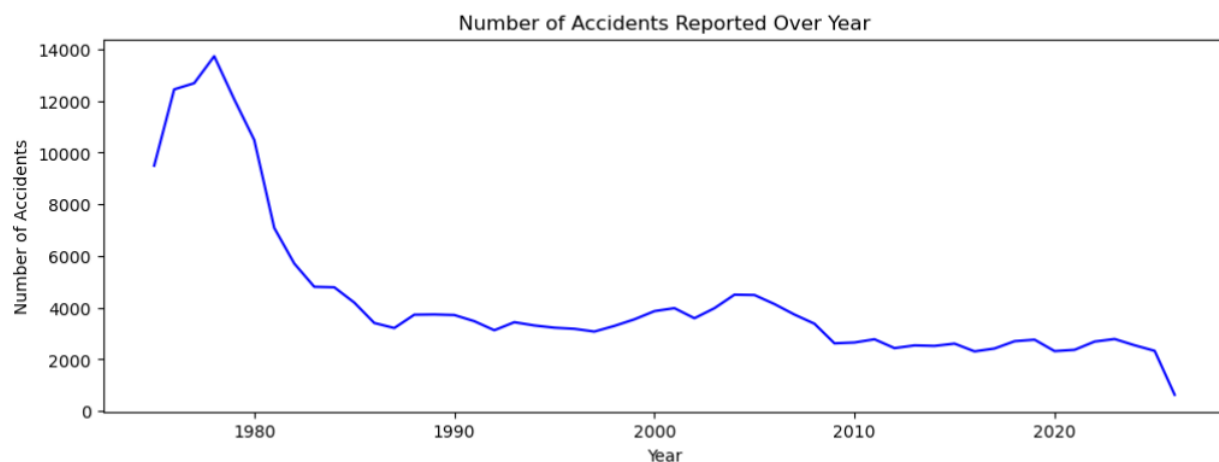
**Dataset:**

The dataset titled “Rail Equipment Accident/Incident Data (Form 54)” was pulled from Data.Transportation.gov on June 30<sup>th</sup>, 2026 (Office, 2021). The Department of Transportation does not require licensing to access publicly available data. The data source is an actively updated dataset. All forms are submitted “30 days after the expiration of the month in which the accident/incident occurred” by the railroad company (6180.54 - Rail Equipment Accident/Incident, 2025).

The dataset contains information regarding the initial accident and additional findings after further investigation. The data pulled on the specific date mentioned previously contained 224,700 records with 155 features, ranging from 1975 to 2026. From reviewing the feature dictionary available on the site, many features share similar information, but in various formats (e.g., Weather Condition Code and Weather Condition). In addition, some features were added in 2011 with no prior entries. The raw dataset was uploaded to a PostgreSQL database and pulled into a Jupyter Notebook via SQL commands for exploration. After data cleaning, the 23 features with 159,995 records were used for preprocessing. *Totaldamagecost*, *Totalpersonskilled*, and *Totalpersoninjured* were used for target engineering risk score (*risk\_score\_yeo*). Data was split into train, validation, and test sets in chronological order to prevent data leakage for target engineering and model building. Preprocessing phase concluded with 121 features.

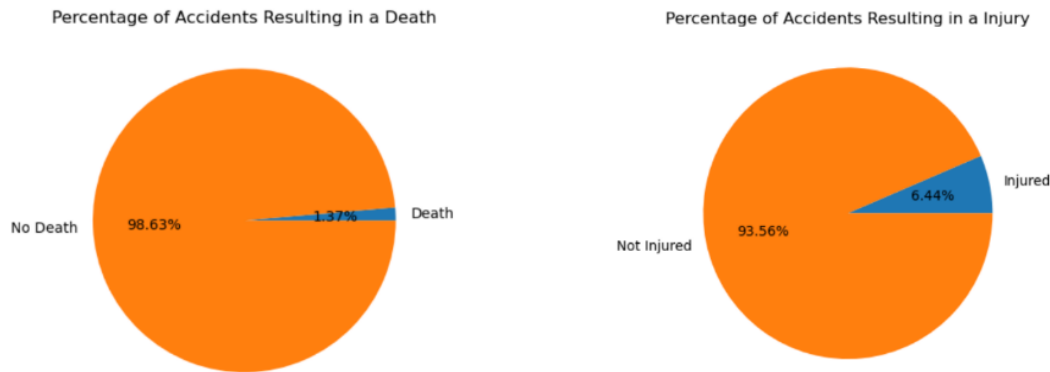
**EDA Part 1: Raw Data Exploration**

Figure One: Total Accidents by Year



The total number of accidents was plotted by year. There was a massive spike in reporting between 1975 and 1980, with the remaining years stabilizing between 3,000 and 5,000 records until 2026. The reason for the minimal 2026 records is that the records for this year are still being added.

Figure Two: Percentage of Death and Injury Records Reported



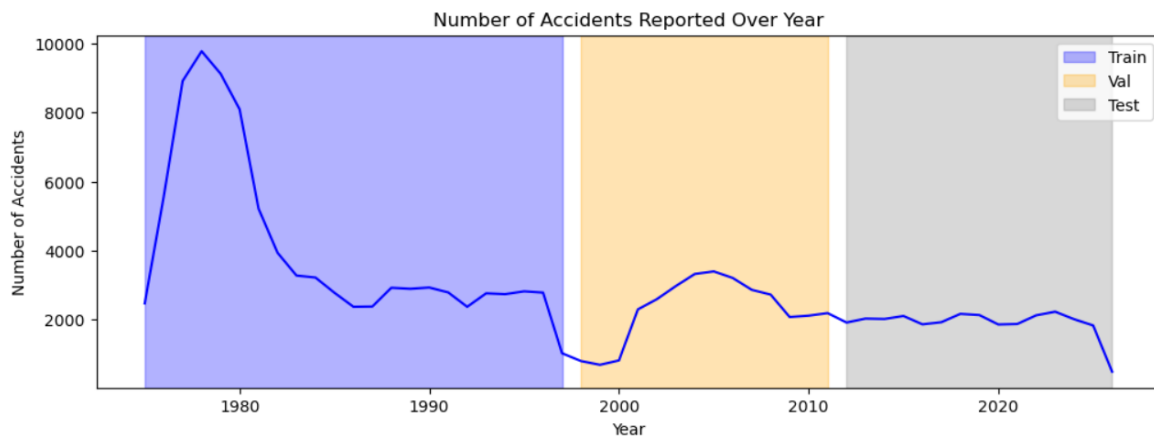
In the dataset, the ratio between accidents reported involving death and injuries was examined and converted into a percentage. Using the feature *Total Persons Killed*, records with values greater than 0 scored 1.37%. In comparison, records with *Total Persons Injured* greater than 0 scored 6.44%.

Figure Three: Table Layout of Top 10 Accident Type by Number of Accidents

|   | Accident Type                 | Accident Type Code | num_of_accidents | total_damage | total_killed | total_injured |
|---|-------------------------------|--------------------|------------------|--------------|--------------|---------------|
| 0 | Derailment                    | 1                  | 143717           | 11970763546  | 369          | 12682         |
| 1 | Side collision                | 4                  | 21977            | 1002227506   | 103          | 2549          |
| 2 | Other impacts                 | 12                 | 21953            | 931262737    | 69           | 1777          |
| 3 | Hwy-rail crossing             | 7                  | 11909            | 924579129    | 3029         | 13162         |
| 4 | Other (describe in narrative) | 13                 | 4977             | 348568330    | 44           | 733           |
| 5 | Rear end collision            | 3                  | 4823             | 723695796    | 226          | 8878          |
| 6 | Raking collision              | 5                  | 4471             | 220887894    | 39           | 539           |
| 7 | Obstruction                   | 9                  | 4410             | 368715437    | 130          | 2222          |
| 8 | Fire/violent rupture          | 11                 | 3185             | 227154074    | 11           | 281           |
| 9 | Head on collision             | 2                  | 2359             | 720982239    | 331          | 4270          |

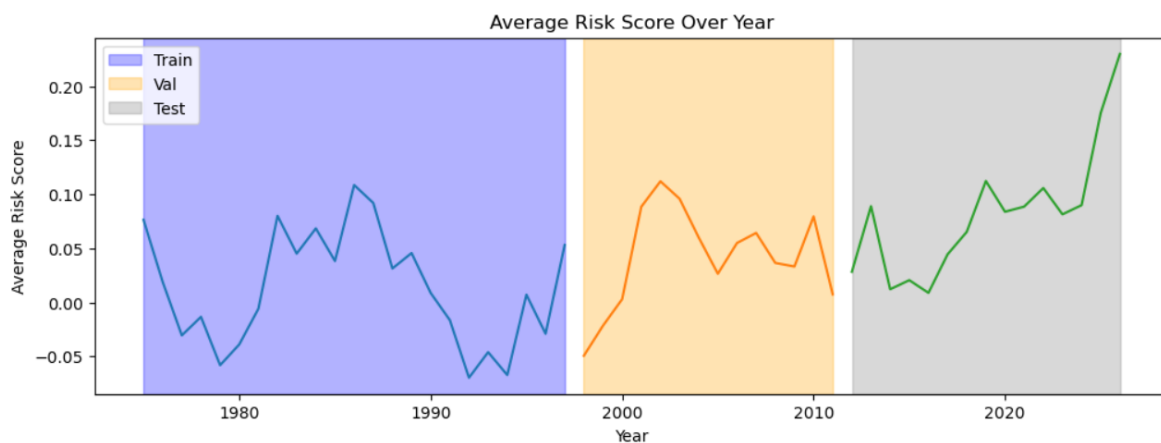
The following features were used to construct the visual: *Accident Type*, *Accident Type Code*, *Total Damage Cost*, *Total Persons Killed*, *Total Persons Injured*. The most reported accident type was “Derailment”, with 143,717 records that had a total financial damage cost of \$11,970,763,546. In comparison, “Hwy-rail crossing” had 11,909 records, but carried the largest number of deaths at 3,029 and injuries at 13,162.

*Figure Four: Number of Accidents Over Time-Range After Cleaning*



After data cleaning, the number of accidents per year was replotted with the train, validation, and test split to evaluate if dropping 70,705 records significantly altered the plot. The most significant change is the dip in records between late the 1990s and early 2000s, which occurred in the validation time range. Fewer records for those years pose potential difficulty in predicting risk score and category.

*Figure Five: Average Risk Score Over Time-Range*



Each year was aggregated by calculating the average risk score for all records within each year. The average score fluctuated sporadically, with years between 2024 and 2026 having a sharp upward trend. Overall, risk score trend per grouping leans toward having a subtle increase over years.

Figure Five: Correlation Heatmap of Target Features for Engineering Risk Score

Before target engineering, a correlation heatmap was generated to check for multicollinearity. The highest correlation score was between *damage\_ratio* (*totaldamagecost* engineered score) and *totalpersonskilled* at 0.31. These features were chosen for target engineering because the feature categories represent cost, death, and injuries.

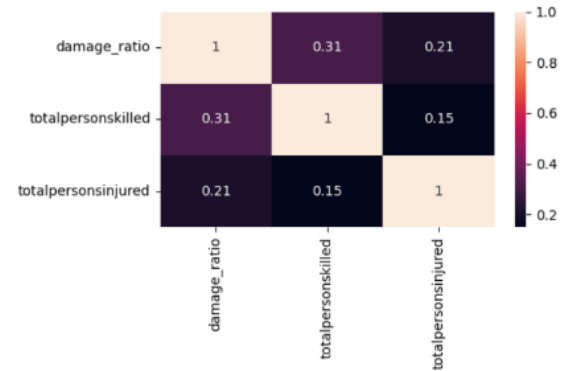
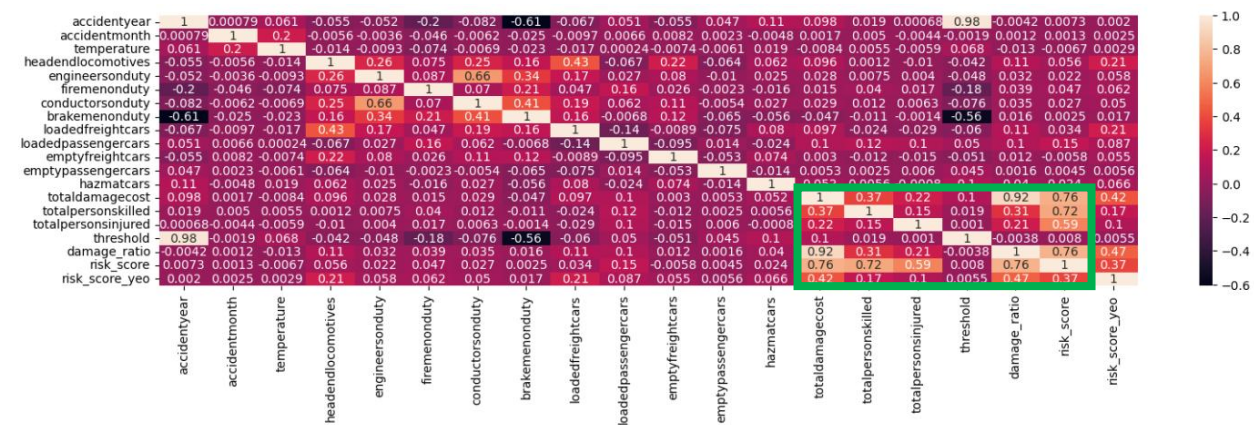


Figure Six: Heatmap Correlation to Risk Score



The numeric features' correlation with the risk score (*risk\_score\_yeo*) suggests that none of the features planned for modeling indicate a strong relationship. Excluding the features used for target engineering, the strongest signal amongst the features came from *headendlocomotives* and *loadedfreightcars*. The green square indicates features used for target engineering.

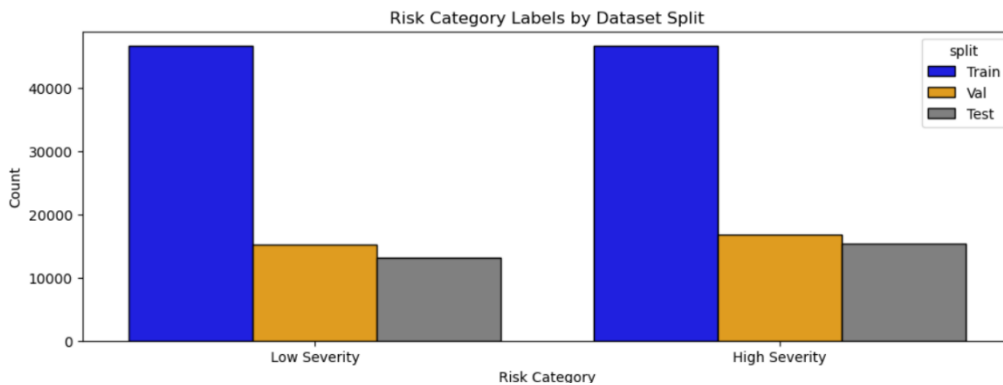
Figure Seven: Categorical Feature Cardinality Check

Before OneHotEncoder was applied, categorical features were evaluated for high cardinality. The highest feature was *stateabbreviation* at 50 unique values. Because no high values over 100 were present, frequency counting was not applied.

```
Categorical Number of Unique Values
accidenttype : 12
stateabbreviation : 50
district : 8
visibilitycode : 4
equipmenttype : 9
weatherconditioncode : 6
tracktype : 4
passengerstransported : 2
traindirection : 4
trackclass : 8
```

Each categorical feature, except for *stateabbreviation* due to having the highest number of unique values, was expanded to evaluate the average risk score in connection to each category's unique value. The feature *accidenttype* had the most discoveries, with “Head-on collision” (0.823), “Hwy-rail crossing” (0.792), and “RR grade crossing” (0.891) having the highest magnitudes above the average set at 0. This indicated that these three types of accidents will play an important role in predicting a higher risk score.

Figure Eight: Risk Category Amount per Dataset



The *qcut* method splits the training set into the two severity classes. However, it does not guarantee that the validation and test sets will also be balanced due to the method's design. The validation set has more than 1,392 High Severity cases and 2,082 Low Severity cases compared to the test set.

## Data Preprocessing:

Preprocessing of the records included correcting the uppercase and additional spaces between words in the feature names and dropping the five duplicated records because the *incidentkey* and *reportkey* were identical. To assess missing values, a data frame was constructed to look at features that are missing more than 10% of their records, concluding 48 features.

To streamline the process, Claude AI was given the following prompt: “Based on the list I have provided, call out features that could be beneficial in terms of being used to predict risk of an initial accident.” Based on Claude's output and further exploration of the feature list, 23 features were selected for modeling, with three specifically (*totaldamagecost*, *totalpersonskilled*, and *totalpersonsinjured*)

selected for target variable engineering. Further data cleaning involving the 23 features was conducted to correct outliers and missing values using the value count method in pandas. Features used for target variable engineering were not cleaned to maintain data accuracy when producing a risk score. The features *accidentyear* and *year* repeated the same information, but in different formats (e.g., *accidentyear*: 88, *year*: 1988). Therefore, *year* was transformed into *accidentyear*. All records with missing values remaining were automatically dropped. The final cleaned dataset size contained 153,995 records and 26 features.

To maintain the dataset's chronological order, *accidentyear* was ordered and broken down into train, validation, and test sets based on the cutoff years, engineered using the numpy quantile method with values set at 0.6 and 0.8. Concluding with the following time range per split:

- 1) Train Time: (1975-1997)
- 2) Validation Time: (1998-2011)
- 3) Test Time: (2012-2026)

The command “is\_monotonic\_increase” was used by *accidentyear* to confirm that the data was still in chronological order per dataset. OneHotEncoder was applied to categorical features by fitting it to the training set and transforming the validation and test sets afterward. All categorical features in their original form were dropped.

StandardScaler was applied to the continuous features in the datasets except for month. The scaler was fitted to the training set and then applied to the validation and test sets. The equation for StandardScaler is a record's actual value minus that feature's mean, divided by the feature's standard deviation.

Month follows a circular format. To correctly scale month, functions from numpy called “sin” and “cos” were applied to generate two additional features per dataset titled *month\_sin* and *month\_cos*. Each function used 2 times the value of pi divided by the number of months, which was multiplied afterward by each record's month value. After handling month appropriately, *accidentmonth* was dropped from all three datasets.



The final feature count used for modeling was 121 features. To prepare data for modeling, each dataset was broken up into three components: the features used for modeling, the target variable for the regression model, and the target variable for the classification model.

### **Feature Engineering Risk Score and Risk Category:**

Each feature set aside for target engineering was examined using the described method from pandas to access outliers, revealing a *totaldamagecost* maximum value of \$3,153,875, with a mean of \$84,584 and a standard deviation of \$346,223. To address the concern of currency inflation over 51 years, each year's minimum threshold amount used for reporting was collected through investigation on the FRA site, mapped to each record based on its corresponding *accidentyear* in each dataset (*Form 6180.54 Threshold - Dot-Fra Search Results*, 2026). To produce *damage\_ratio*, *totaldamagecost* was divided by its corresponding *accidentyear* threshold value.

This ratio represents how many times over that year's minimum threshold a specific accident cost is. Important note: the years 2019, 2023, and 2024 had no evidence online for their specific threshold values. Therefore, those years were automatically filled in based on the previous year's threshold value. After transformation, *damage\_ratio* replaced *totaldamagecost* as a feature for the risk score.

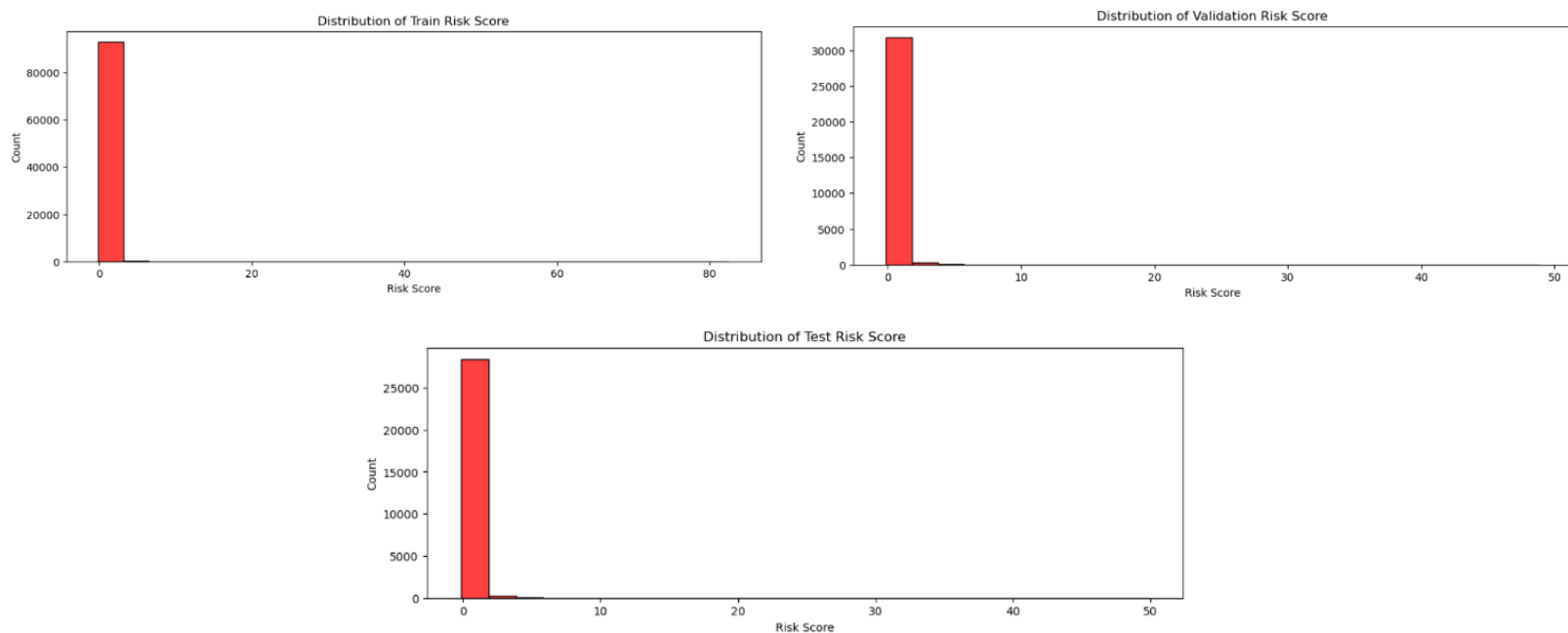
When constructing the risk score, it is not correct to assume that all features are weighed equally. However, there is no evidence found at the time of this report to suggest a manual weight for each feature. Through the webpage titled "Performing Principal Component Analysis (PCA) to Determine Weights for Index Indicators," PCA loading weights were applied to each feature for target engineering (2023).

Before using PCA, StandardScaler from SKlearn was applied to the training set and transformed onto the validation and test sets to maintain a consistent scale amount of all three features. Using the first principal component, which examines the best combination of each feature in capturing the most variation in the data, the absolute loading value was calculated and transformed into a weighted percentage that adds up to 1. The following are the weights of all three features:

- 1) *damage\_ratio*: 0.369
- 2) *totalpersonskilled*: 0.346
- 3) *totalpersonsinjured*: 0.285

To calculate the risk score, each feature's value per record was multiplied by its assigned weight and summed across all three features to produce the target variable *risk\_score*. This same step was applied to the validation and test sets.

Figure Nine: Train-Val-Test Risk Score Distribution

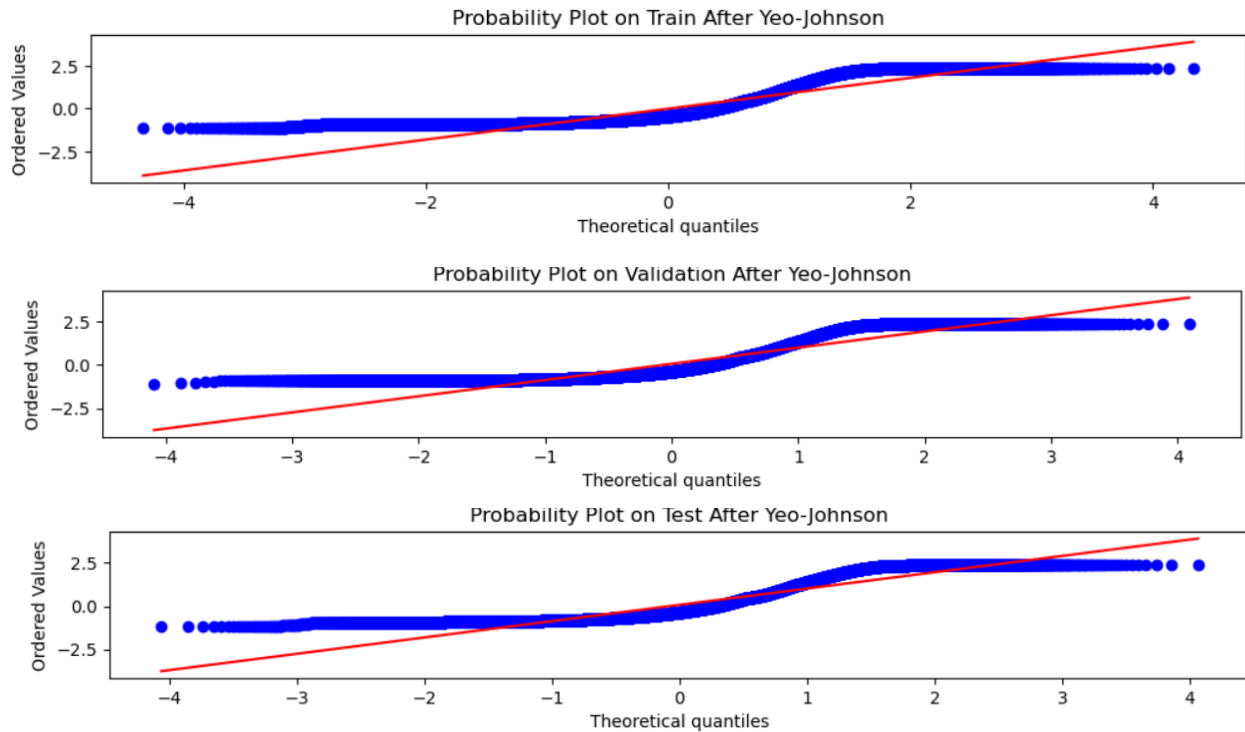


The risk score distribution revealed an extreme right-skew shape for all three datasets. This is the result of keeping the outlier values in all three features, as manually removing the outliers would make an inaccurate risk model. In addition, the `scipy.stats` function call `skew` was applied to the training set and received a score of 37.35. To be considered normally distributed, the skew value should be at zero.

To address the right-skew, Yeo-Johnson Power Transformation was applied to make the data appear more bell-shaped by stretching and/or compressing values with the `lambda` parameter, which is automatically selected. Lambda values closer to 0 follow a log transformation. Values at 1 have no change, and values at 2 experience heavy compression. This method was chosen over Box-Cox or Log Transformation because risk scores had zeros present within all three datasets. Yeo-Johnson used

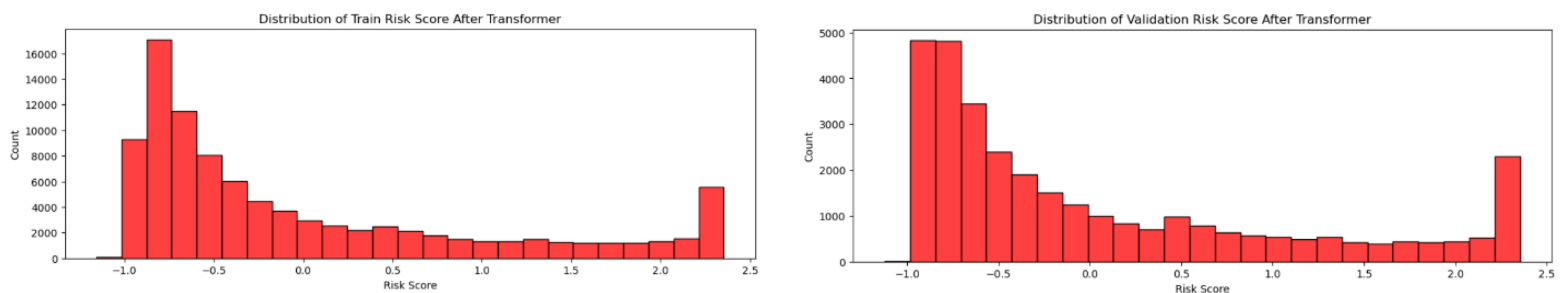
Sklearn's preprocessing technique PowerTransformer to standardize the data distribution to have a mean of 0 and a standard deviation of 1.

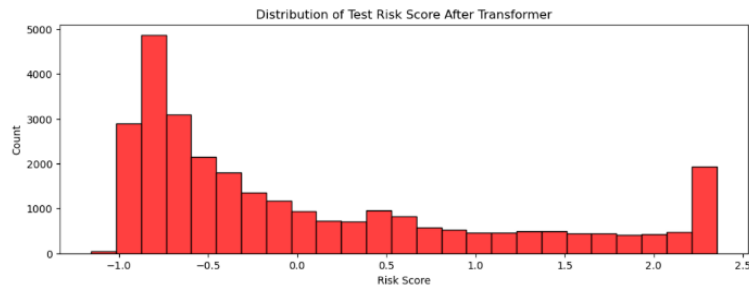
Figure Ten: Yeo-Johnson Visual Transformation



The transformation revealed that it corrected a large portion of the risk score skewness. However, the plateauing values at both ends indicate repeating values, as shown by examining the value count at the low and high ends of each dataset. Therefore, there is still a small margin of skewness left within the data, as evidenced by its skew score 1.14. The new name for the risk score after transformation is *risk\_score\_yeo*.

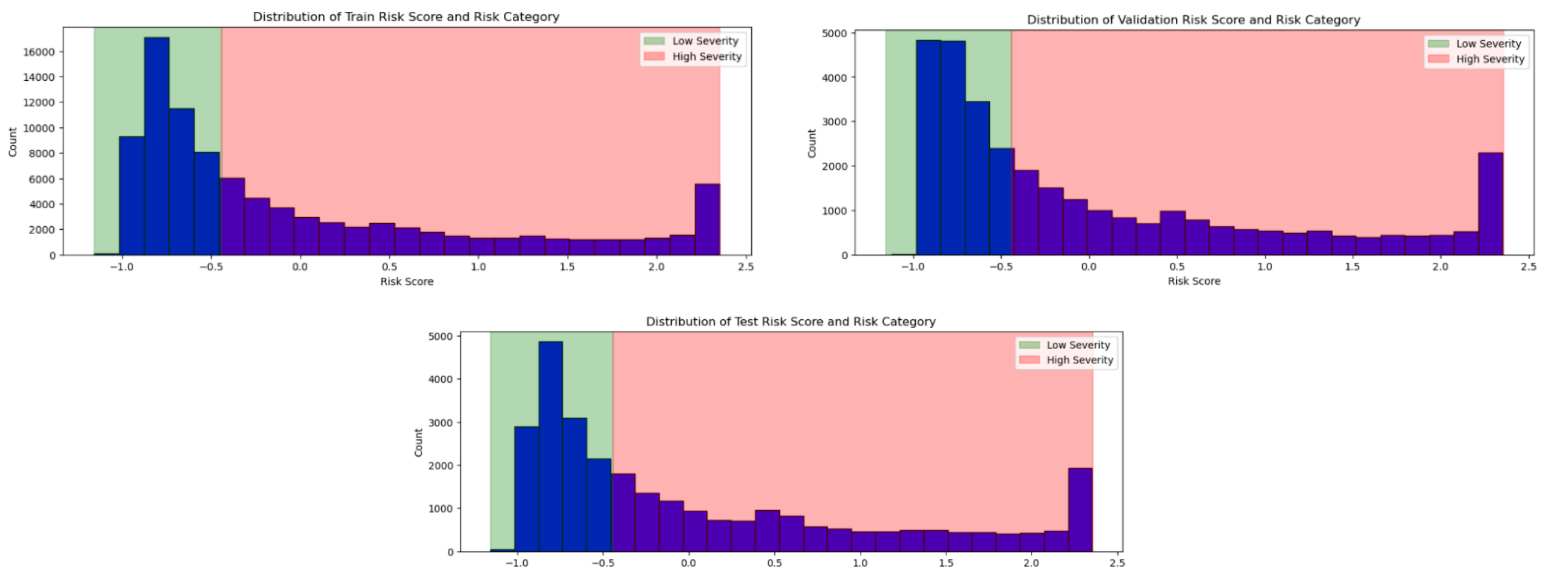
Figure Eleven: Distribution of Risk Score After Transformation





The above image shows right skewness within the scoring range. However, it is less extreme compared to the figure 9. To interpret *risk\_score\_yeo*, PowerTransformer converted the range into standard normal form. Therefore, values at 0 represent the average of the training dataset, and values at 2 are considered higher risk because they are two standard deviations from the mean. This does not indicate that the score is two times more severe than 0. In addition, the high spikes at the far end of each visual confirm the repeating values from Yeo-Johnson.

*Figure Twelve: Distribution of Risk Score and Risk Category*



Regarding target engineering risk category (*risk\_category*), there was no evidence available to suggest what the default cutoff between Low Severity and High Severity should be. To categorize *risk\_score\_yeo*, the pandas method called *qcut* was applied to manually split the training set into two groups based on the idea of having an equal number of records in both classes. This transformation was then applied to the validation and test sets by using the “*retbins*” parameter and the pandas method called *cut*. The cut parameter “*include\_lowest*” was used to prevent the lowest values in validation and test from being cut. The cutoff value between both classes was set to -0.443.

**Methodology:**

All data modeling and evaluation were conducted in Python using pandas, NumPy (numpy), scikit-learn (sklearn), XGBoost, SciPy, Matplotlib, and Seaborn. The experimental pipeline began with the assumption that risk score and risk category could easily be calculated/distinguished by models like Linear Regression and KNN. Before implementing the models, feature correlation and importance were evaluated. Regarding feature correlation in connection to the target variable *risk\_score\_yeo*, the strongest positive correlation was *tracktype\_Main* at 0.399, while the strongest negative was *trackclass\_1* at -0.371.

To further explore feature importance, DecisionTreeRegressor and DecisionTreeClassifier with a random state of 42 for reproducibility revealed that *tracktype\_Main*, *temperature*, and *loadedfreightcars* were the three most dominant features for both models, but in different orders. It is important to note that though feature importance scores appear small, ranging between 0.160 and 0.080, it does not indicate that they are weak. Feature importance was extracted using the command “feature\_importances\_” on the training data when fitting the models.

To predict the risk score, three Linear Regression models were examined to determine effectiveness. Linear regression is used to predict a continuous score by assuming a linear relationship between input features and the target. It does this by minimizing the squared error between the predicted and actual values. The first approach used the base model without any parameters, which yielded a validation RMSE score of 0.889 and R-squared score of 0.252. Ideally, because the risk score is transformed into standard normal form with a mean of 0 and a standard deviation of 1, the validation RMSE should be lower than 1. A value of 1 would indicate there is no difference from predicting the mean every time. In comparison, the higher the R-squared value is, the more the model was able to explain the pattern within the data. The base model proves that it could only explain roughly 25% of the data.

One addition was to implement a sample weight to place more importance on recent years versus those from far behind. The weight, titled “year\_weight”, was created using the training data’s feature

*accidentyear*. By taking each record's year, subtracting it from the minimum value (1975) and adding a value of 1, the model improved slightly, with a validation RMSE score of 0.886 and R-squared score of 0.257.

In connection with this improvement, the best number of features was selected in the hope of improving the model even more. To do this, the GridSearchCV method from Sklearn was paired with a pipeline setup. The pipeline specifically used SelectKBest from Sklearn with the parameter score function set to "f\_regression" to select the best number of features. This function tests each feature against the target variable one by one. Therefore, it cannot evaluate features that, combined with others, produce a better linear relationship. The cross-validation technique TimeSeriesSplit with a value of 5 was implemented. TimeSeriesSplit was chosen because it splits the data in chronological order, adding additional data as each fold increases.

To score the grid search, Sklearn's scoring parameter "neg\_root\_mean\_squared\_error" was utilized. To evaluate the RMSE, the scores produced from the grid search were converted from their negative values to positive for easier interpretability. The grid search was then implemented on the training data and produced the best number of features, which was 116, a 5-feature difference from what was already implemented. The new list of features from SelectKBest scoring was fitted on the training set and then used to transform the validation and test sets to match. Linear Regression was retested with the newly selected features and "year\_weights," and no significant change was found. Cross-validation using TimeSeriesSplit with the scoring methods of "neg\_root\_mean\_squared\_error" and "r2", showed that the model performed consistently across folds but had weak predictive power overall.

Due to the weak signals, Linear Regression was ruled out as an effective method, as its methodology assumes that the risk score follows a linear path. Therefore, XGBoost Regressor, a non-linear model that utilizes numerous decision trees to predict, was implemented. XGBoost begins with a single baseline prediction applied to all records, then calculates the residual. New decision trees are trained to predict this residual, and a learning rate scales each tree's output before being added to the running prediction. This process repeats, with residuals shrinking over each tree, until a stopping

condition is met. XGBoost's equation is calculated by taking the sum of the loss function set by the parameter "Objective" plus the penalty terms that prevent decision trees from becoming too complex.

To determine the optimal combination of parameters, a grid search was implemented. Grid search was performed using the original dataset format, with no weights or the best number of features selected. The grid search used the same cross-validation and scoring methods as in previous linear models. The following parameters and their values were pulled from the grid search and implemented into the model testing:

- 1) Objective='reg:squarederror': Indicated that this is a regression problem by minimizing the squared error.
- 2) Random\_state = 42: Set for reproducibility.
- 3) Colsample\_bytree = 0.7: For each decision tree, randomly select 70% of features to use. Used to reduce overfitting by adding randomness into calculation. Requires more decision trees and could lead to underfitting if set too low. [search values: 0.7, 0.9]
- 4) Subsample = 0.7: For each decision tree, randomly select 70% of records to use. Same benefits and issues as with colsample\_bytree. [search values: 0.7, 0.9]
- 5) Learning\_rate = 0.01: Scaling of the correction from each tree before adding to the predictive total. Lower threshold values require more trees, which in turn results in slow performance because it takes more conservative steps. [search values: 0.01, 0.05]
- 6) Max\_depth = 7: Maximum level each decision tree is allowed to grow before being cut off. Prevents overfitting but could lead to underfitting by missing important signals if set too small. [search values: 6, 7]
- 7) N\_estimators = 700: Maximum number of decision trees that can be built. A larger number of estimators capture more complex patterns in the data but requires more training time and risks overfitting. [search values: 700, 800, 900]

After grid search and testing the model, the validation RMSE was 0.889, a 0.003 score difference compared to the Linear Regression model with “year\_weights” and 116 selected features. However, the R-squared score improved to 0.293, a 0.037 increase. This indicates that the XGBoost Regressor was able to capture more of the data variance.

Using the same cross-validation setup as in the Linear Regression portion, the score per fold continues to stay stable, but is still weak at predicting the risk score. This indicates that instead of having more data, there need to be more features that strongly align with predicting the risk score.

Figure Thirteen: XGBoost Learning Curve

Using the same learning curve parameters from before, the visual indicates that the gap between train and validation is slowly closing as more data is fed into the prediction. However, given that training is increasing while validation remains within the 0.85-0.90 RMSE range, this could also indicate that a) a sign of overfitting, and b) there are not enough features with strong enough signals to produce predictive power.

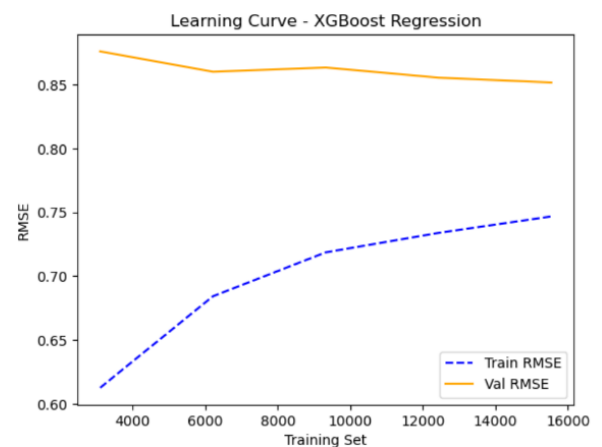


Figure Fourteen: XGBoost Actual vs Predicted

The actual vs predicted plot confirms that the model is still struggling to grasp a strong enough signal to predict the risk score effectively. However, with an R-square score of 0.293, the model is still making real predictions.

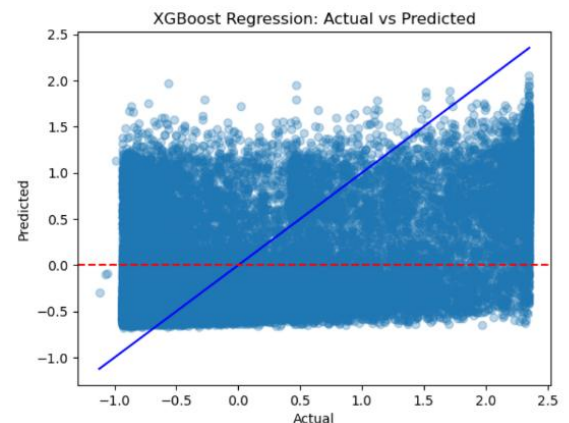
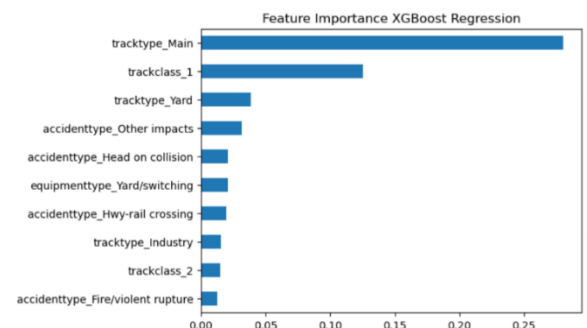


Figure Fifteenth: XGBoost Feature Importance

Limiting to the top 10 strongest features, the number one feature used in the model was *tracktype\_Main*, followed by *trackclass\_1*. The low scores are because all 121 features used for modeling received a feature importance score, and they all collectively add up to 1.





After assessing the regression models, the KNN classification model was used to predict risk categories. KNN is a non-linear, distance model that calculates the space between a point and its neighbors using formulas such as Euclidean or Manhattan, then classifies that point based on the majority class among its nearest neighbors. The first approach used the default KNN parameters, with the addition of setting `n_neighbors` to 5.

The base KNN model produced an accuracy score of 0.60. However, High Severity had a recall score of 0.54, indicating that it missed roughly 46% of these actual cases. Missing High Severity cases is much more detrimental compared to wrongly classifying one. Therefore, High Severity recall will be the most important metric to evaluate whether the model is improving. In addition, F1-score will be the final determinant as it focuses on both precision and recall simultaneously. In the KNN model, the F1-score for High Severity was 0.59 and for Low Severity was 0.62.

To determine the optimal KNN model, grid search paired with a pipeline setup was used. The pipeline was constructed to select the optimal number of features used. KNN is a distance-based model, so removing noisy features improves model calculation. The scoring method selected for the `“score_func”` parameter was `“f_classif.”` Like `“f_regression”`, but this method focuses on classification models as it compares each feature between each class. Grid search used the same cross-validation as in previous models but replaced the scoring parameter with `“f1_macro.”` This scoring parameter calculates the F1-score per class and then averages them together. F1-macro offers equal treatment between classes, so one does not outshine the other.

After processing the grid search, the parameter values were implemented into the model. The following are the parameters used for modeling:

- 1) `K = 100`: Number of features used for modeling. [search values: 20, 50, 100]
- 2) `N_neighbors = 100`: Number of nearest neighbors used to classify each point. [search values: 20, 50, 100]
- 3) `p = 1`: Manhattan Distance [search values: 1, 2] [formula:  $d(x,y) = \sum |x_i - y_i|$ ]

- 4) Weights = 'distance': Closer points have more influence compared to further points. [search values: 'uniform', distance']

The model produced an accuracy score of 0.64. High Severity recall fell to 0.49 from 0.54, but its precision increased to 0.73, resulting in an F1-score of 0.58. Low Severity increased its precision to 0.59 and recall to 0.80, resulting in an F1-score of 0.68. Though the model improved in other areas, it missed more than 50% of High Severity cases.

KNN without a selected number of features was also applied through a grid search with the same parameters, cross-validation technique, and scoring method. The only parameter change was reassigning weights to "uniform." The grid search values were implemented into the model and produced the same scores as the previously tuned KNN model, except for High Severity F1-score, which increased by 0.01.

Due to the weak performance of the High Severity class, an XGBoost Classifier model was implemented to evaluate improvement. Before conducting a grid search and implementing the model, the target variable for classification had to be transformed into binary format using sklearn's preprocessing technique LabelEncoder. The LabelEncoder allows the XGBoost Classification model to produce a classification prediction with the objective parameter set to "binary:logistic". High Severity was given the label 0, while Low Severity was given 1.

Grid search was formatted with the same parameters as used for XGBoost Regressor, with the only difference coming from the scoring method set to F1 Macro. Through grid search, the returned parameters were identical to those established for XGBoost Regressor. Unlike XGBoost Regressor, which predicts a continuous value, XGBoost Classifier converts its final score into a probability and assigns a class label based on a threshold using log loss.

Using the parameters established through grid search, the model classification report was produced. With an overall accuracy of 0.66, the model improved compared to the KNN model by 0.02 points. High Severity (label 0) recall was 0.57, a massive improvement from 0.49 with KNN. F1-score for High Severity increased to 0.63, while Low Severity stayed at 0.68. To confirm model performance across TimeSeriesSplit folds, cross-validation was conducted with the scoring method "f1\_macro" for

consistency and produced stable results with small deviations between folds. However, the results remain in the 0.66 to 0.68 range, hinting that increasing data does not improve model learning.

Figure Sixteen: XGBoost Classification Learning Curve

The visual of the XGBoost Classifier indicates that the model is learning, due to the gap between train and validation shrinking. However, the gap overall is still large, indicates that the model has reached the maximum it can learn with the data.

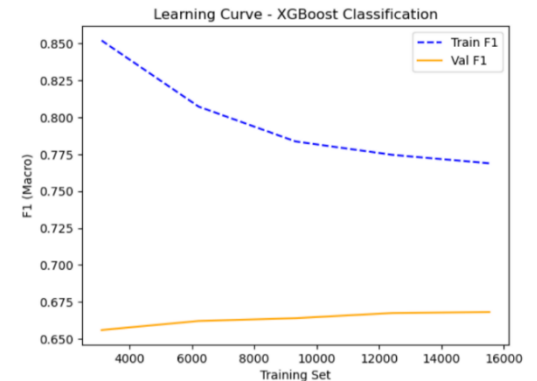
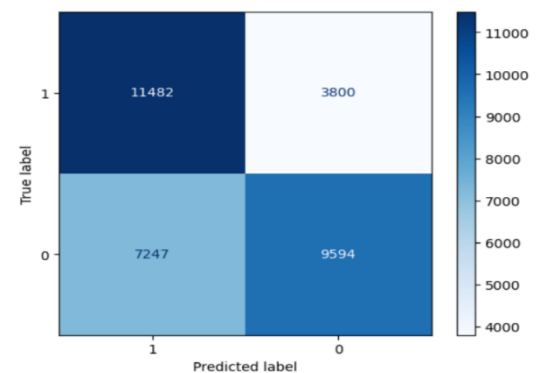


Figure Seventeen: XGBoost Classification Confusion Matrix

The confusion matrix confirms the recall score for High Severity (label 0), as it is capturing more than 50% of the data labeled 0. It is still missing a large portion. Low Severity captures a large majority of its labeled 1 target, with the targets slipping through as labeled High Severity, which is not a concern at this current moment.



After exploring the models for Regression and Classification, the XGBoost Regressor and Classifier have been selected as the two models used for testing. To compare these models, Sklearn models DummyRegressor and DummyClassifier were used. DummyRegressor, with a strategy of “mean,” predicts the average for every record. DummyClassifier, with a strategy of “stratified,” generates predictions randomly while keeping the training set’s class distribution, so it is a fair comparison.

## Model Evaluation:

For model evaluation, it is broken into Regression and Classification components. For Regression, the evaluation metrics used were RMSE and R-squared. RMSE (Root Mean Squared Error) measures the average size of prediction errors, in the same format as the target. To determine success, lower RMSE scores mean better prediction performance. To calculate RMSE, the actual values are subtracted from predicted values, and the resulting difference is squared. Each error score is summed and

divided by the total number of records, producing the MSE score. The last step is to take the square root of that score to convert it back to the original unit.

R-squared indicates how much better one model is compared to guessing the average every time. With a range of 0 to 1, scores closer to 1 are considered favorable. To calculate R-squared, the model subtracts each predicted value from its actual value, squares the difference, and sums them together to get the residual error. The same process is repeated using the difference between each actual value and the mean to get the total variance. Finally, the calculated error is divided by the variance value and then subtracted from one to get the final score.

For classification, the evaluation metric was F1-macro. The reasoning is that F1-macro combines each class's individual F1-score and then averages them. To calculate F1-score, the class's precision and recall are multiplied, and then multiplied by 2. This is followed by dividing that value by the value produced when adding precision and recall together. This is for one class. So, the process is repeated for the second class. The final step is to add both classes' individual F1-score and divide by the number of classes. The F1-macro score metric is favorable because it weighs both classes equally, so neither can outperform the other.

For statistical testing, a paired t-test and bootstrap resampling with replacement were used. The cross-validation technique of TimeSeriesSplit with a fold set to 5 was used. The paired t-test was used to examine whether the difference between each XGBoost and Dummy model was significantly different than 0. Bootstrap with 10,000 iterations and replacements provides an additional check, but in an independent setting. A paired t-test should have a high t-statistic and a p-value below 0.05 to be considered statistically significant. Bootstrap-calculated results should have a confidence interval that does not include 0 to indicate a difference between the two models.

## **Results and Discussion:**

For model comparison, regression and classification XGBoost models were compared to two naïve models corresponding to each prediction strategy. For regression, DummyRegressor with a strategy

set to mean was implemented, which predicts the mean every time. For classification, DummyClassifier with a strategy set to stratified was used, allowing predictions to follow the class distribution and then guessing the majority class.

*Figure Eighteen: Regression Model Evaluation*

|   | Model                        | Val RMSE | R-squared |
|---|------------------------------|----------|-----------|
| 0 | Linear Regression            | 0.889    | 0.252     |
| 1 | + year_weights               | 0.886    | 0.257     |
| 2 | + SelectKBest (116 features) | 0.886    | 0.257     |
| 3 | XGBoost Regressor (Tuned)    | 0.889    | 0.293     |

To determine which model to use against the naïve model, each model was evaluated on the validation set's RMSE and R-squared. Between the four models, the RMSE score remained in the 0.886 to 0.889 range. Therefore, the decision was made to pivot using specifically R-squared as the determining metric. XGBoost Regressor tuned was chosen due to having the highest R-squared at 0.293, explaining 29% of the data variance.

*Figure Nineteen: Classification Model Evaluation*

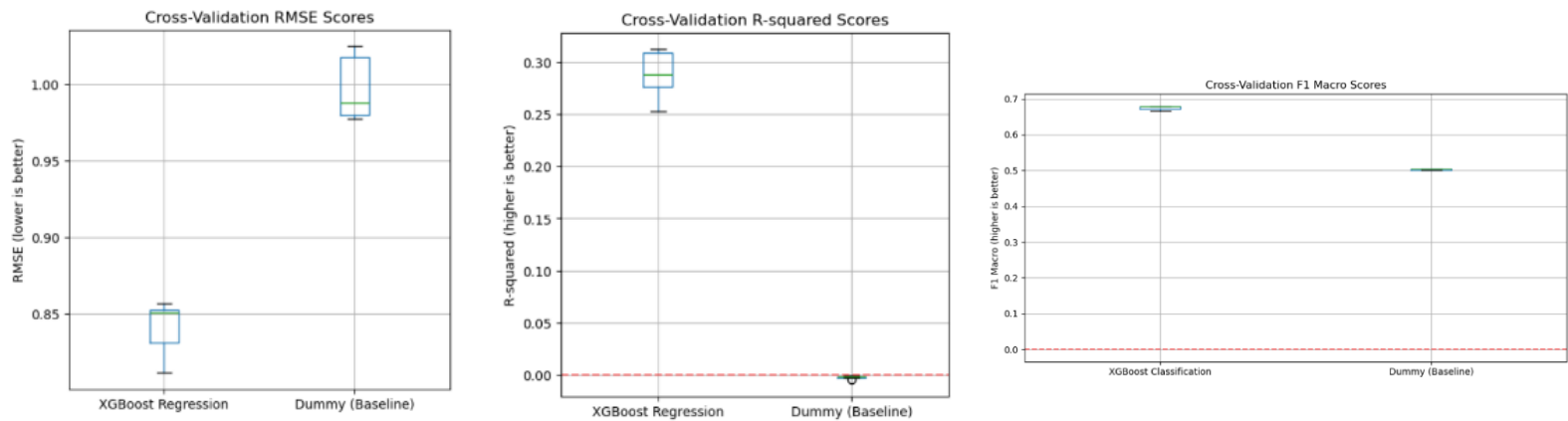
|   | Model                      | Accuracy | High Severity Recall | High Severity F1-score | Low Severity F1-score |
|---|----------------------------|----------|----------------------|------------------------|-----------------------|
| 0 | Base KNN (n_neighbors=5)   | 0.60     | 0.54                 | 0.59                   | 0.62                  |
| 1 | Tuned KNN                  | 0.64     | 0.49                 | 0.59                   | 0.68                  |
| 2 | XGBoost Classifier (Tuned) | 0.66     | 0.57                 | 0.63                   | 0.68                  |

For classification, because High Severity recall is a critical metric for risk models, the highest score was used to determine chosen model. Accuracy was included because it is the first metric evaluated when producing a classification report. F1-score for both classes was also added as the classification evaluation metric using F1-macro, illustrating a complete image. XGBoost Classifier tuned was chosen due to High Severity recall scoring 0.57, only missing 43% of cases.

Before conducting statistical testing, XGBoost and Dummy models' performance was first calculated via cross-validation on the training set, using the same setup in previous models. Below are the mean for each fold calculated and their standard deviation:

| Model              | Scoring Metric | Mean   | STD   |
|--------------------|----------------|--------|-------|
| XGBoost Regressor  | Neg RMSE       | 0.841  | 0.017 |
| Dummy Regressor    | Neg RMSE       | 0.997  | 0.020 |
| XGBoost Regressor  | R-squared      | 0.287  | 0.022 |
| Dummy Regressor    | R-squared      | -0.002 | 0.001 |
|                    |                |        |       |
| XGBoost Classifier | F1 Macro       | 0.675  | 0.005 |
| Dummy Classifier   | F1 Macro       | 0.503  | 0.002 |

*Figure Twenty: XGBoost and Dummy Model Cross-Validation Performance Visualized*



For the three visuals, the overarching conclusion is that the XGBoost models outperformed the dummy models by scoring metric. At the same time, the strategies applied to each dummy model performed as expected. DummyRegressor with a strategy set to mean shows RMSE remaining around 1, and R-squared started at 0. DummyClassifier with a strategy set to stratified reveals 0.50 due to catering to the class distribution via scoring, which was equal due to the qcut transformation step.

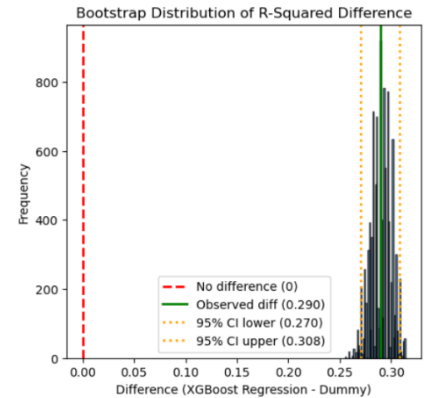
### Statistical Analysis:

The paired t-test was implemented on both the regression and classification models. Although RMSE was used for completeness, it was not used as the concluding decision. For R-squared, the test produced a t-statistic of 26.26 and a p-value of 0.0000125. With the high t-statistics and low p-value, it is

concluded to regret the null hypothesis, stating that the XGBoost Regressor is statistically different than the Dummy model. To confirm this through randomness, bootstrap was implemented to construct the 95% confidence interval between the two models.

*Figure Twenty-One: Bootstrap with 10,000 Iteration for Regression*

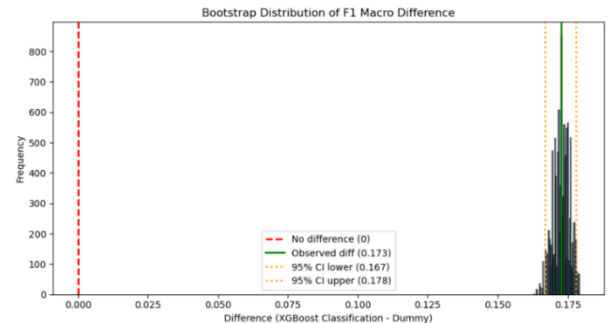
The mean R-squared difference between the two models was 0.290, with a confidence interval between 0.270 and 0.308. Using the visual as an additional confirmation, the confidence interval does not contain 0. Therefore, agree with the previous statement to reject the null hypothesis.



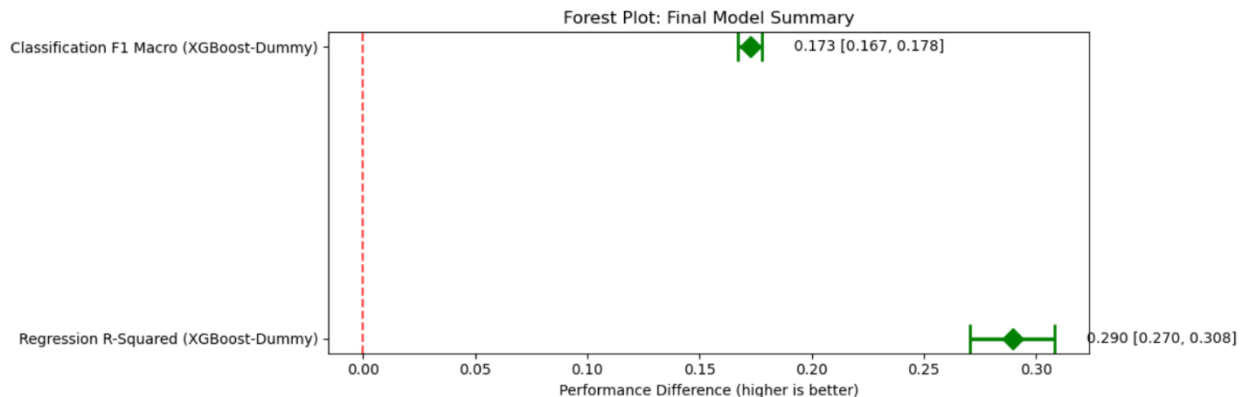
For F1-macro, the test produced a t-statistic of 54.85 and a p-value of 0.0000007. Similarly to the previous regression score, the conclusion can be made to reject the null hypothesis as there is a statistical difference between the XGBoost Classifier and the naïve model. In addition, bootstrap was used to confirm the results:

*Figure Twenty-Two: Bootstrap with 10,000 Iteration for Classification*

The mean F1 macro difference between the two models was 0.173, with a confidence interval between 0.167 and 0.178, confirming the presence of 0 is absent. Therefore, rejecting the null hypothesis.



*Figure Twenty-Three: Forest Plot of Model Performance*

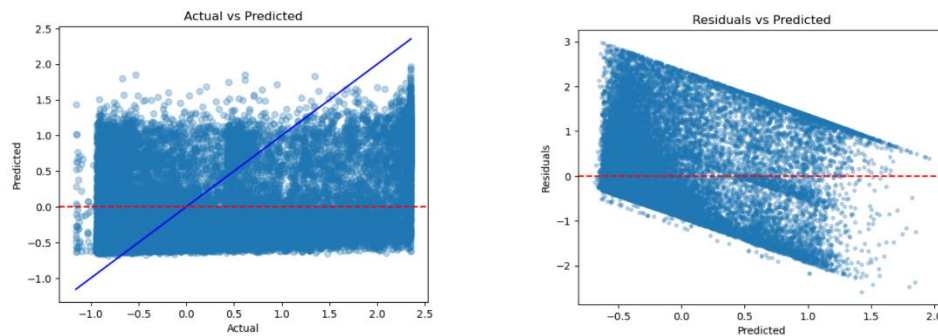


XGBoost for regression and classification using R-squared and F1-macro as scoring metrics does indicate beating the baseline models as neither touches nor crosses the red line. Therefore, we can state that the difference between the two models is statistically significant, supporting the alternative hypothesis that it is possible to produce a risk score and category using pre-incident features.

## Error Analysis

Both XGBoost models were chosen as the final version for statistical testing. Each model was implemented on the unseen test set to evaluate its performance.

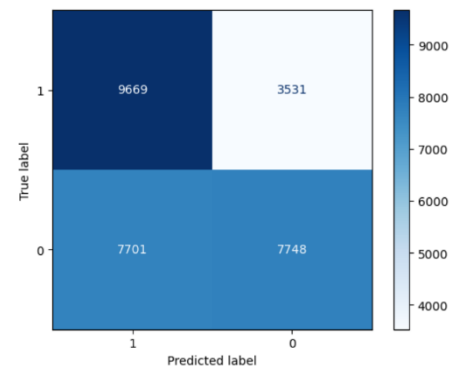
*Figure Twenty-Four: Test Set XGBoost Regressor Results*



For regression, the RMSE score was 0.934 and R-squared was 0.293. The RMSE score increased compared to the validation set by 0.045. However, it remained below 1. R-squared remained consistent between the two tests. Visually, the test sets produced similar results as the validation set, indicating that much of the data could not be captured.

*Figure Twenty-Five: Test Set XGBoost Classifier Results*

For classification, the recall score for High Severity was 0.50, capturing 50% of the High Severity cases, but also missing the remaining 50%. This is also indicated via the confusion matrix. The final F1 scores were 0.58 for High Severity and 0.63 for Low Severity. Combined, the F1 macro was 0.61, 0.05 points below validation.





The XGBoost Regressor model's performance on the test set reveals that most of the data points fail to trace the diagonal line as they follow the path of a cylinder shape. However, there is a slight bend in the cluster overall shape, hinting at capturing a small portion of the data. This is confirmed with the R-squared score being 0.293. However, a large majority of the variance is still unexplained.

The XGBoost Classifier model's errors are more significant. Shown in the confusion matrix (figure seventeen), the model correctly identified a large majority of the Low Severity cases. However, High Severity struggled to capture a large majority of its cases, which is the model's biggest error.

The possible reasoning for these errors is that there are not enough features used for modeling to carry a strong enough signal to reliably assist each model in producing higher scores. So instead of adding additional data, more features or feature interactions would provide more knowledge to each model.

## **Interpretation**

The results support the hypothesis that pre-incident features from the FRA Form 6180.54 data source provide enough information to establish risk scores and classifications for railroad accidents more effectively than a dummy model. Both models outperformed the naïve model set. XGBoost Regressor scoring an R-squared of 0.287 compared to -0.02, and XGBoost Classifier an F1-macro of 0.675 to 0.503. Paired t-test and bootstrap resampling both confirmed that each model was statistically significant.

Regarding both models' performances, there was no evidence of either underfitting due to the learning curves showing training moving towards the positive direction per scoring metric as more data was implemented. It could be argued that the model was overfitting slightly due to the gap persisting in both models. For evaluating the test set, the stronger model would be XGBoost regression regarding generalization. The changes between validation and test were minor compared to the classification version, as all metrics dropped below what was set for validation. Specifically, High Severity recall took the biggest drop, weakening the model's core ability to predict the more important class.

## Strength

Regarding the strength in the methodology, the main praise is for maintaining a rigorously safe data leakage environment. Due to engineering a target variable using statistical methods, maintaining honesty in calculating the risk score was crucial. In addition, continuous alignment was maintained following a chronological order of the data. This ranged from manually splitting and confirming chronological order was maintained, to applying `TimeSeriesSplit` as the cross-validation method.

The methodology also highlighted the benefit of using PCA for feature engineering as it provides an evidence-based method of assigning weights, and Yeo-Johnson for addressing the skewness as result of keeping outliers to produce more realistic risk models. The two statistical tests were implemented to confirm that the results from feature engineering and modeling implementation were genuine and did not result from random chance, confirming the rejection of the null hypothesis.

## Limitations

The question constantly asked was whether these features could produce a strong enough signal in predicting and classifying risk. Although the statistical tests confirmed this question, the low R-squared and recall scores suggest that a large portion of the data is missing. Therefore, additional features provided outside of the dataset could hold stronger predictive power than using Form 6180.54 alone. In connection with the previous sentence, further exploration into what information an executive knows at the exact moment of an accident could uncover more beneficial information for future work.

The other limitation is that there is no evidence provided to suggest what the actual cutoff limit between Low and High Severity is. This report implements `qcut` as the placeholder, but having a stronger cutoff limit with statistical research backing its claim could improve the model in terms of authenticity. Plus, this would allow for transforming the risk score into an easier-to-interpret value than what is currently provided. This would prevent professionals from generating the wrong conclusion on the information generated.

Finally, a limitation of this analysis is that the alternative models (KNN and Linear Regression) were not put through the same statistical testing. So, it remains unknown whether their performance differences from XGBoost are statistically meaningful or due to chance.

## Biases

One bias that exists in calculating risk is that severity has no natural limit. Therefore, assuming something to be 100% risk is not a fair statement. Using `qcut` as the threshold method between classes assumes there is a maximum value. Therefore, additional research is needed to address this concern, as the current method offers a controlled environment when risk is chaotic. The other bias is time. Due to the shift in accident patterns and reporting metric over time, risk is constructed and evaluated in a manner that does not assume every record is on a similar plan. Producing *damage\_ratio* helped address the minimum threshold per year. There are still additional factors to consider.

## Societal Impact

These models are intended to help determine a starting position for developing a risk mitigation strategy. They are not intended to be the final decision-maker because additional components of each accident are not included in the model. Thus, due to the performance metrics being lower than anticipated, with the classification model missing 43% of High Severity cases, it is encouraged that professionals conduct research to confirm or reject the results produced.

## Conclusion

To assess risk, the question was posed whether pre-incident features from the FRA Form 6180.54 can help professionals determine accident severity involving equipment. The argument was that pre-incident features provide enough statistical power to establish risk scores and classifications for railroad accidents more effectively than a dummy model. Through a paired t-test and bootstrap resampling with

replacement, it was concluded that both XGBoost Regressor and XGBoost Classifier outperformed the dummy model by a statistical margin.

Despite this significance, these models still missed a large amount of data. Therefore, they are intended to only be used as supporting materials and not as a final decision maker. For future improvement, the main objective is to establish a more credible threshold cutoff between the two severity classes and address the drift in severity over time. In addition, consider adding NLP or LLM extraction to utilize the narrative column in the original dataset.

## Reference

Office, F. (2021). Rail Equipment Accident/Incident Data (Form 54). In *Transportation.gov*.

[https://data.transportation.gov/Railroads/Rail-Equipment-Accident-Incident-Data-Form-54-/85tf-25kj/about\\_data](https://data.transportation.gov/Railroads/Rail-Equipment-Accident-Incident-Data-Form-54-/85tf-25kj/about_data)

*6180.54 - Rail Equipment Accident/Incident*. (2025). Usdot.

<https://railroads.dot.gov/safety-data/forms-guides-publications/forms/618054-rail-equipment-accidentincident>

*Handbook on Constructing Composite Indicators METHODOLOGY AND USER GUIDE*. (n.d.).

Retrieved August 3, 2026, from

[https://www.oecd.org/content/dam/oecd/en/publications/reports/2008/08/handbook-on-constructing-composite-indicators-methodology-and-user-guide\\_g1gh9301/9789264043466-en.pdf](https://www.oecd.org/content/dam/oecd/en/publications/reports/2008/08/handbook-on-constructing-composite-indicators-methodology-and-user-guide_g1gh9301/9789264043466-en.pdf)

Performing principal component analysis (PCA) to determine weights for index indicators.

(2023). In *ArcGIS Blog*. <https://www.esri.com/arcgis-blog/products/api-python/analytics/performing-principal-component-analysis-pca-to-determine-weights-for-index-indicators>

*form 6180.54 threshold - dot-fra Search Results*. (2026). Usa.Gov.

<https://search.usa.gov/search?query=form+6180.54+threshold&affiliate=dot-fra>

**Data Source Link:**

- [Rail Equipment Accident/Incident Data \(Form 54\) | Department of Transportation - Data Portal](#)

**Reproducibility Library Links:**

- [StandardScaler — scikit-learn 1.9.0 documentation](#)
- [PowerTransformer — scikit-learn 1.9.0 documentation](#)
- [OneHotEncoder — scikit-learn 1.9.0 documentation](#)
- [PCA — scikit-learn 1.9.0 documentation](#)
- [TimeSeriesSplit — scikit-learn 1.9.0 documentation](#)
- [GridSearchCV — scikit-learn 1.9.0 documentation](#)
- [LinearRegression — scikit-learn 1.9.0 documentation](#)
- [KNeighborsClassifier — scikit-learn 1.9.0 documentation](#)
- [DummyRegressor — scikit-learn 1.9.0 documentation](#)
- [DummyClassifier — scikit-learn 1.9.0 documentation](#)
- [SelectKBest — scikit-learn 1.9.0 documentation](#)
- [learning\\_curve — scikit-learn 1.9.0 documentation](#)
- [https://xgboost.readthedocs.io/en/stable/python/python\\_api.html](https://xgboost.readthedocs.io/en/stable/python/python_api.html)
- [ttest\\_rel — SciPy v1.18.0 Manual](#)
- [skew — SciPy v1.18.0 Manual](#)
- [pandas.qcut — pandas 3.0.5 documentation](#)
- [pandas.cut — pandas 3.0.5 documentation](#)